In [2]:
```
#project Student marks analysis

#import necessary libraries
import pandas as pd
import  matplotlib.pyplot as plt

#create a simple tabular dataset
data = {
    'Student': ['Kazi', 'Islam', 'Monira', 'Bushra', 'Noor'],
    'Math': [60, 88, 69, 59, 90],
    'Science': [58, 79, 69, 96, 80],
    'English': [77, 87, 57, 74, 99]
}

#Create dataframe
df = pd.DataFrame(data)

#Display table
df
```

Out[2]:

| | Student | Math | Science | English |
|---|---|---|---|---|
| **0** | Kazi | 60 | 58 | 77 |
| **1** | Islam | 88 | 79 | 87 |
| **2** | Monira | 69 | 69 | 57 |
| **3** | Bushra | 59 | 96 | 74 |
| **4** | Noor | 90 | 80 | 99 |

In [3]:
```
#calculate average marks
#Calculate average marks for each student
df['Average'] = df[['Math', 'Science', 'English']].mean(axis = 1)

#Display updated DataFrame
df
```

Out[3]:

| | Student | Math | Science | English | Average |
|---|---|---|---|---|---|
| **0** | Kazi | 60 | 58 | 77 | 65.000000 |
| **1** | Islam | 88 | 79 | 87 | 84.666667 |
| **2** | Monira | 69 | 69 | 57 | 65.000000 |
| **3** | Bushra | 59 | 96 | 74 | 76.333333 |
| **4** | Noor | 90 | 80 | 99 | 89.666667 |

In [4]:
```
#Assign grades
#Function to assign grades
def assign_grade(avg):
    if avg >= 90:
```

```
        return 'A'
    elif avg >= 80:
        return 'B'
    elif avg >= 70:
        return 'C'
    elif avg >= 60:
        return 'D'
    else:
        return 'F'

#Apply the function to calculate grades
df['Grade'] = df['Average'].apply(assign_grade)

#Display DataFrame with grades
df
```

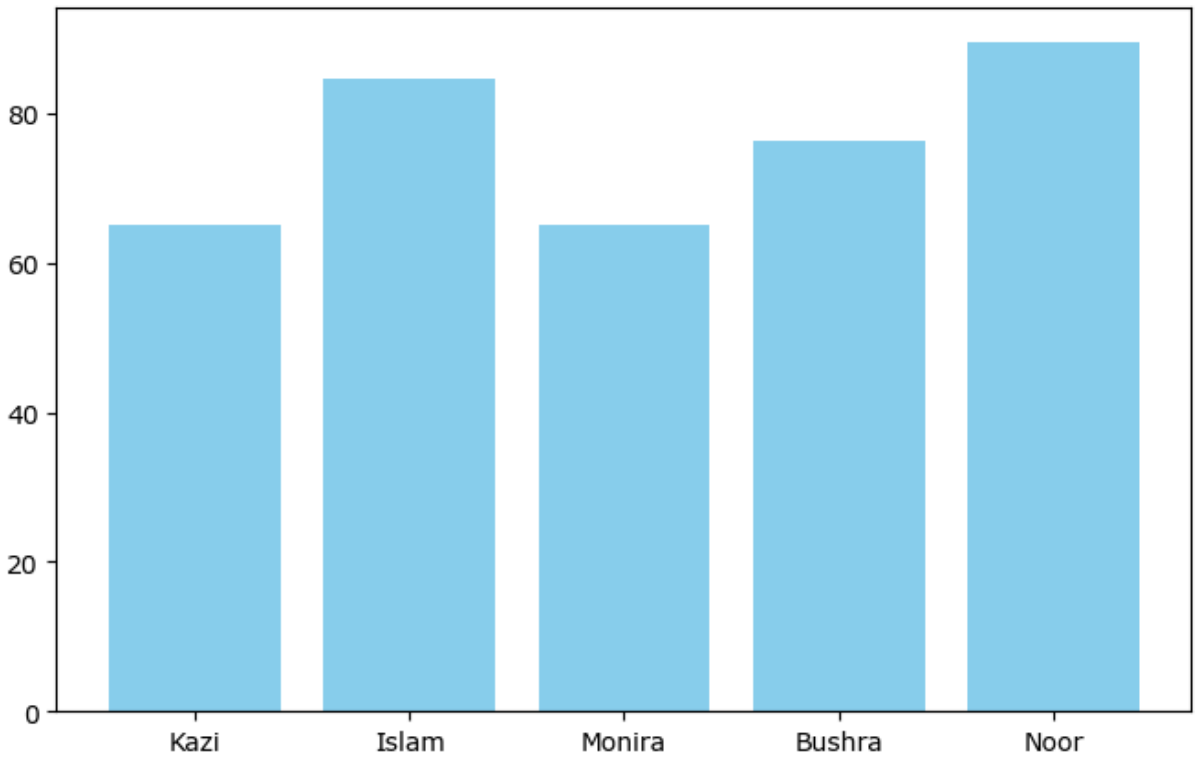
Out[4]:

| | Student | Math | Science | English | Average | Grade |
|---|---|---|---|---|---|---|
| **0** | Kazi | 60 | 58 | 77 | 65.000000 | D |
| **1** | Islam | 88 | 79 | 87 | 84.666667 | B |
| **2** | Monira | 69 | 69 | 57 | 65.000000 | D |
| **3** | Bushra | 59 | 96 | 74 | 76.333333 | C |
| **4** | Noor | 90 | 80 | 99 | 89.666667 | B |

In [5]:

```
#Basic visualization
#Plot average marks of students

plt.figure(figsize=(8,5))
plt.bar(df['Student'], df['Average'], color = 'skyblue')
plt.xlable('Students')
plt.ytable('Average Marks')
plt.show()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[5], line 6
      4 plt.figure(figsize=(8,5))
      5 plt.bar(df['Student'], df['Average'], color = 'skyblue')
----> 6 plt.xlable('Students')
      7 plt.ytable('Average Marks')
      8 plt.show()

AttributeError: module 'matplotlib.pyplot' has no attribute 'xlable'
```




In [14]:
```
#Titanic Data Analysis
#Analysis the Titanic dataset to understand passenger demographics, survival

#import libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

#optional: improve plot style
#sns.set_style('whitegrid')
plt.style.use('seaborn-v0_8-whitegrid')

#load dataset from a csv url
#url = "https://www.kaggle.com/datasets/yasserh/titanic-dataset"
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/tit
df = pd.read_csv(url)

#display first 5 rows
df.head()
```

Out[14]:

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | |
| **1** | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 7 |
| **2** | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | |
| **3** | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 5 |
| **4** | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | |

In [6]:

```
#shape of the dataset (rows, columns)
print("Dataset shape:", df.shape)

#Basic info
df.info()

#Summary statistics for numeric columns
df.describe()
```

```
Dataset shape: (5, 6)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 6 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   Student  5 non-null      object
 1   Math     5 non-null      int64
 2   Science  5 non-null      int64
 3   English  5 non-null      int64
 4   Average  5 non-null      float64
 5   Grade    5 non-null      object
dtypes: float64(1), int64(3), object(2)
memory usage: 372.0+ bytes
```

Out[6]:

| | Math | Science | English | Average |
|---|---|---|---|---|
| **count** | 5.000000 | 5.000000 | 5.0000 | 5.000000 |
| **mean** | 73.200000 | 76.400000 | 78.8000 | 76.133333 |
| **std** | 14.956604 | 14.117365 | 15.6269 | 11.223982 |
| **min** | 59.000000 | 58.000000 | 57.0000 | 65.000000 |
| **25%** | 60.000000 | 69.000000 | 74.0000 | 65.000000 |
| **50%** | 69.000000 | 79.000000 | 77.0000 | 76.333333 |
| **75%** | 88.000000 | 80.000000 | 87.0000 | 84.666667 |
| **max** | 90.000000 | 96.000000 | 99.0000 | 89.666667 |

In [15]:

```
#Handle the missing data
#Check missing values

df.isnull().sum()
```

Out[15]:

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

In [19]:

```
#Fill the missing values with median
df['Age'].fillna(df['Age'].median(), inplace = True)
#df['Age'] = df['Age'].fillna(df['Age'].median())

#Drop rows where embarked is missing
df.dropna(subset = ['Embarked'], inplace = True)

#Optional: Drop Cabin column due to too many missing values
df.drop(columns = ['Cabin'], inplace = True, errors = 'ignore')

# always check columns first
#print(df.columns)

# safe dropping
#df.drop(columns=['Cabin'], inplace=True, errors='ignore')
```

```
/private/var/mobile/Containers/Data/Application/06DFBEBA-74FA-48CE-8D41-EF80
9EE54699/tmp/ipykernel_3213/2354674791.py:2: FutureWarning: A value is tryin
g to be set on a copy of a DataFrame or Series through chained assignment us
ing an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behave
s as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'd
f.method({col: value}, inplace=True)' or df[col] = df[col].method(value) ins
tead, to perform the operation inplace on the original object.


  df['Age'].fillna(df['Age'].median(), inplace = True)
```

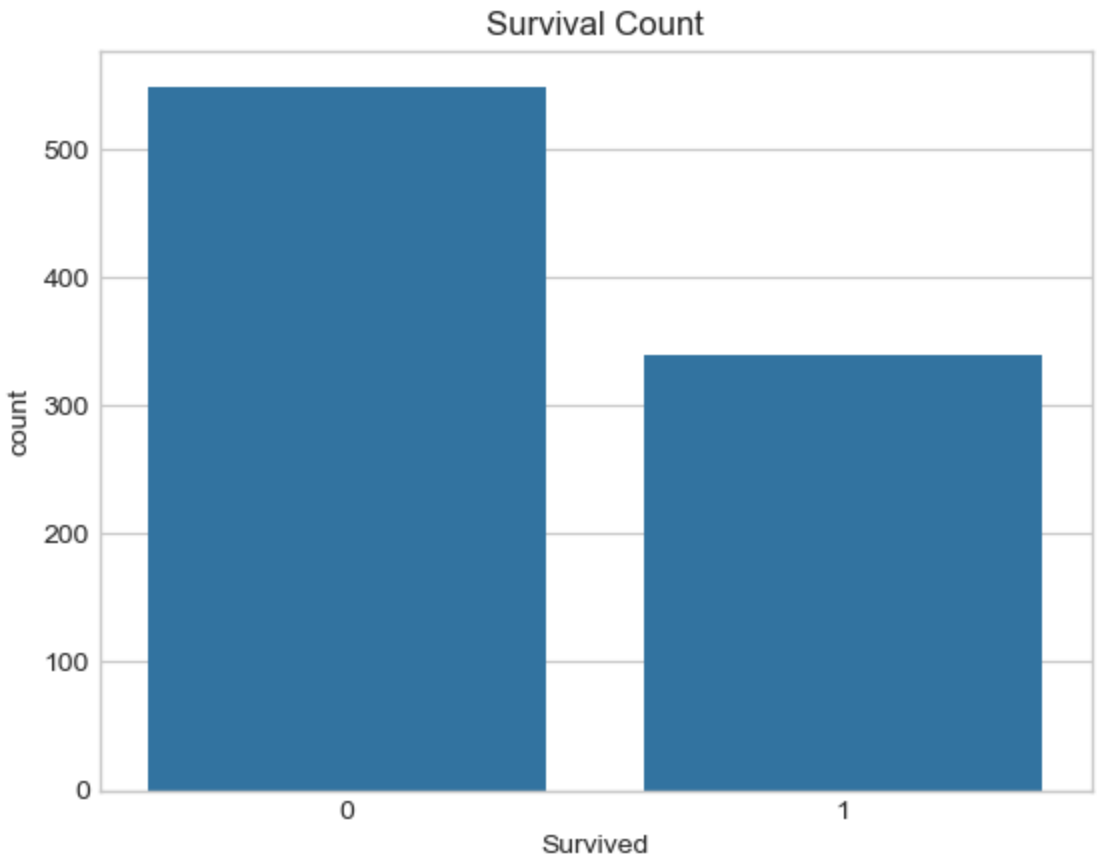
In [20]:
```
#Check missing values
df.isnull().sum()

#Fill missing age values
df['Age'] = df['Age'].fillna(df['Age'].median())

#Drop rows where Embarked is missing
df = df.dropna(subset = ['Embarked'])
```

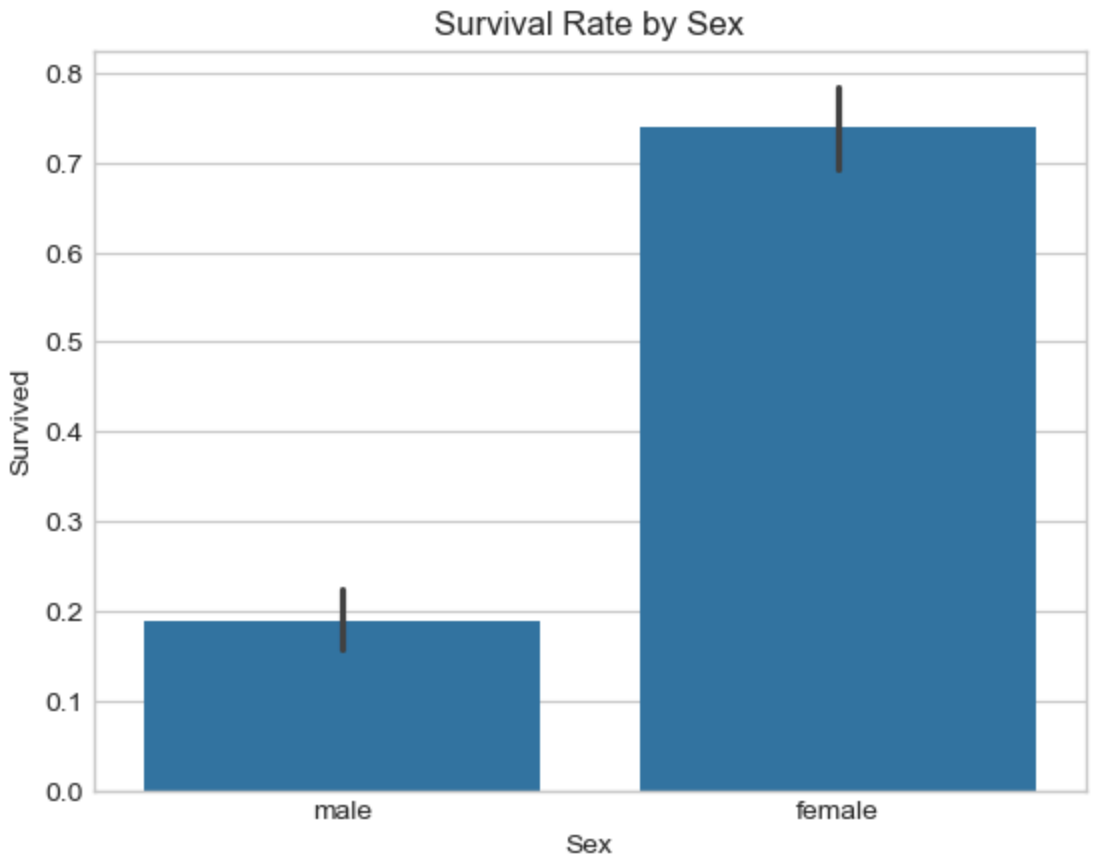
In [21]:
```
#Basic analysis
#How many passengers survived?
survival_count = df['Survived'].value_counts()
print(survival_count)

#Survival rate by sex
print(df.groupby('Sex')['Survived'].mean())

#Survival rate by passenger class
print(df.groupby('Pclass')['Survived'].mean())
```

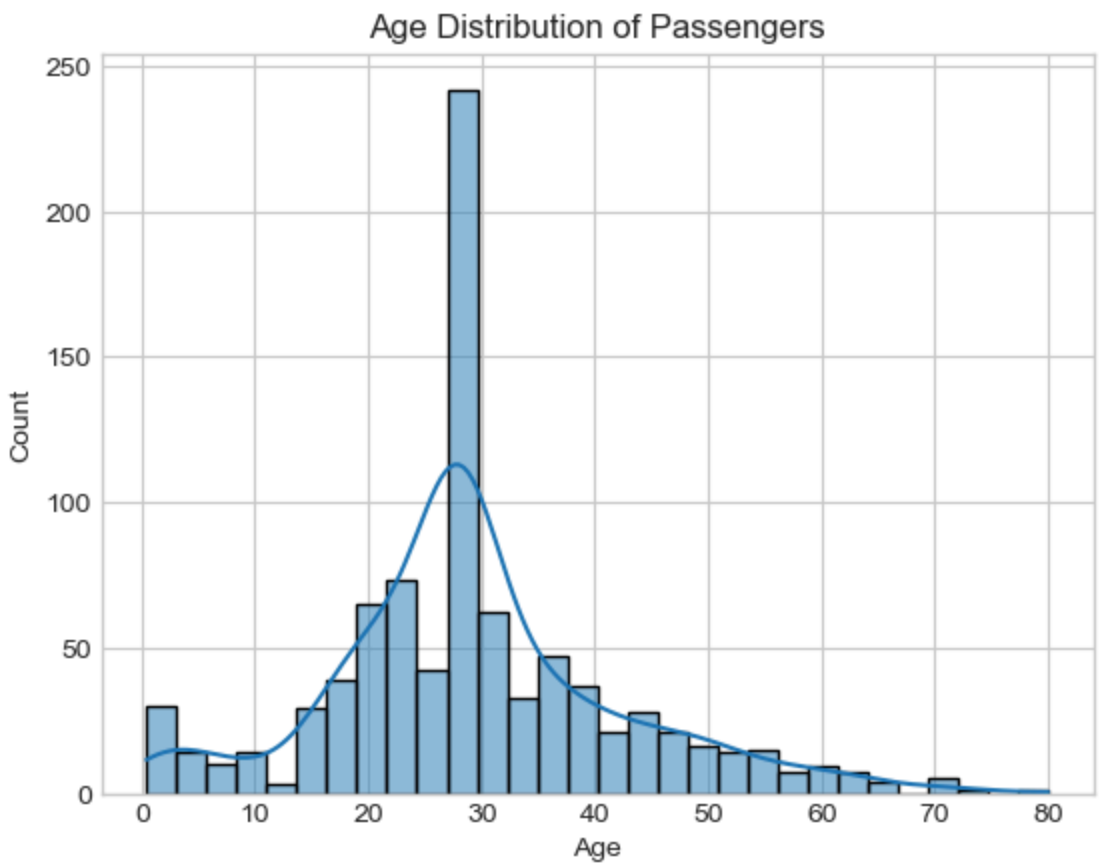
```
Survived
0    549
1    340
Name: count, dtype: int64
Sex
female    0.740385
male      0.188908
Name: Survived, dtype: float64
Pclass
1    0.626168
2    0.472826
3    0.242363
Name: Survived, dtype: float64
```

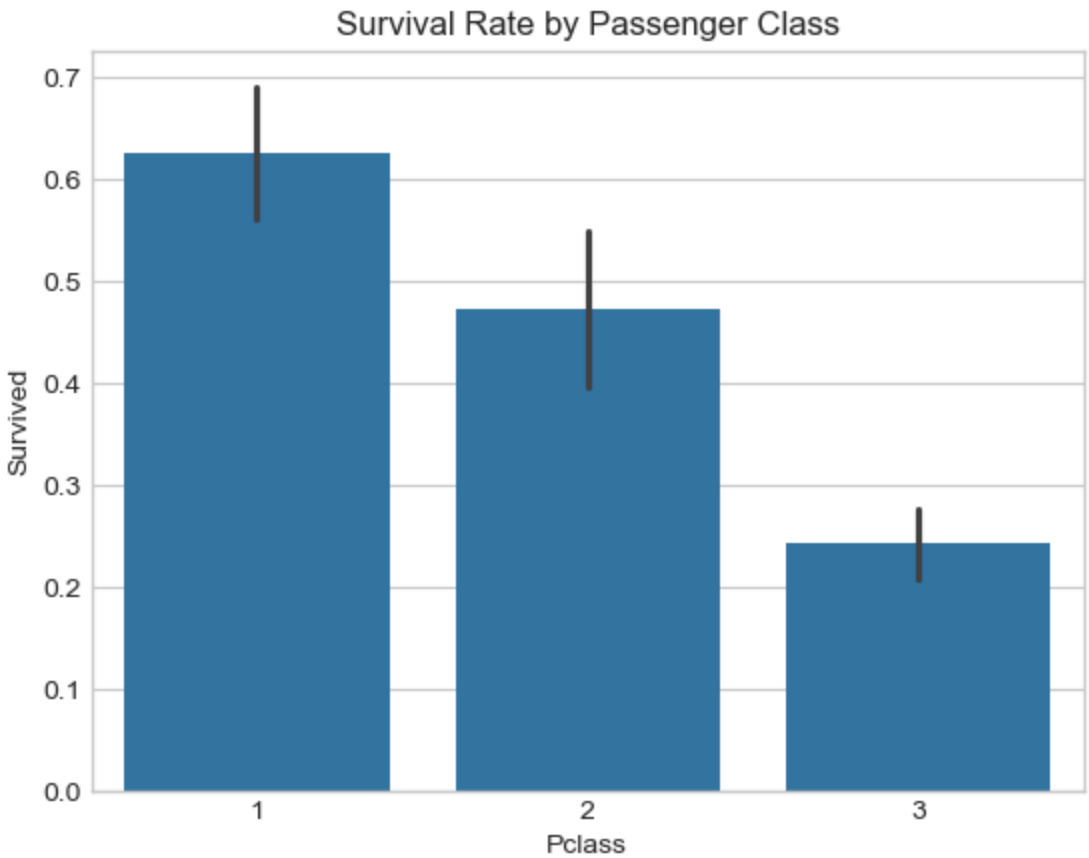
In [22]:
```
#Basic Visualization

#Plot survival counts
sns.countplot(x = 'Survived', data = df)
plt.title('Survival Count')
plt.show()
```

```
#Survival by sex
sns.barplot(x = 'Sex', y = 'Survived', data = df)
plt.title('Survival Rate by Sex')
plt.show()


#Age distribution
sns.histplot(df['Age'], bins = 30, kde = True)
plt.title('Age Distribution of Passengers')
plt.show()

#Survival by Pclass
sns.barplot(x = 'Pclass', y = 'Survived', data = df)
plt.title('Survival Rate by Passenger Class')
plt.show()
```

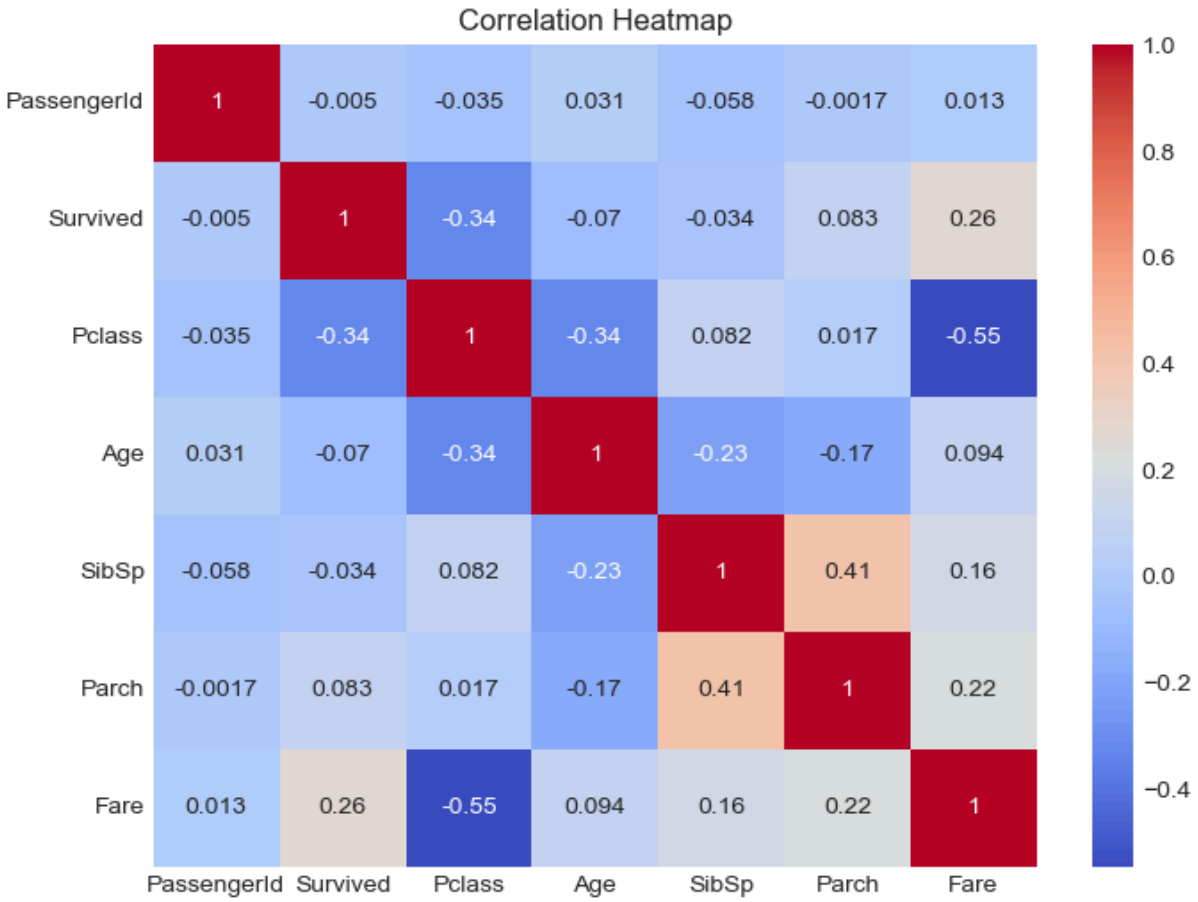



Survival Rate by Sex
Survived
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
male
female
Sex


Age Distribution of Passengers
Count
0
50
100
150
200
250
0
10
20
30
40
50
60
70
80
Age




In [24]:
```
#Correlation Analysis

#Correlation between numeric columns

# Create correlation matrix
corr = df.corr(numeric_only=True)
#Heatmap
plt.figure(figsize =(8, 6))
sns.heatmap(corr, annot = True, cmap = 'coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```


Correlation Heatmap
PassengerId
Survived
Pclass
Age
SibSp
Parch
Fare
1 -0.005 -0.035 0.031 -0.058 -0.0017 0.013
-0.005 1 -0.34 -0.07 -0.034 0.083 0.26
-0.035 -0.34 1 -0.34 0.082 0.017 -0.55
0.031 -0.07 -0.34 1 -0.23 -0.17 0.094
-0.058 -0.034 0.082 -0.23 1 0.41 0.16
-0.0017 0.083 0.017 -0.17 0.41 1 0.22
0.013 0.26 -0.55 0.094 0.16 0.22 1
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

In [ ]: